Задача N1

Пусть имеются две доли, каждая с n вершинами, пронумерованными от 1 до n. Для каждой вершины i левой доли задан отрезок $[l_i, r_i]$, что означает, что вершина i соединена с каждой вершиной правой доли j, для которой выполняется условие:

$$l_i \leq j \leq r_i.$$

Основная идея

Наша цель — выбрать для каждой вершины левой доли (если возможно) вершину правой доли, удовлетворяющую условию, при этом каждая вершина правой доли может быть выбрана не более одного раза. Для этого:

1. Пусть множество $R = \{1, 2, \ldots, n\}$ содержит все свободные вершины правой доли.
2. Отсортируем вершины левой доли по возрастанию r_i. Такой порядок позволяет на каждом шаге выбирать вершину правой доли, которая удовлетворяет условию и имеет минимальный номер, что минимизирует потенциальный конфликт при выборе для последующих вершин.
3. Для каждой вершины i из отсортированного списка находим минимальное $j \in R$, для которого $j \geq l_i$ и одновременно $j \leq r_i$. Если такое j найдено, добавляем ребро (i, j) в паросочетание и удаляем j из R.

Псевдокод алгоритма

Algorithm 1 Максимальное паросочетание в двудольном графе с интервальными ребрами

Вход: Массивы $l[1 \ldots n]$ и $r[1 \ldots n]$
Выход: Множество пар $M \subset \{(i, j)\}$
$R \leftarrow \{1, 2, \ldots, n\}$ ▷ Множество свободных вершин правой доли
Сформировать массив пар $\{(i, l_i, r_i) : i = 1, \ldots, n\}$
Отсортировать массив по неубыванию r_i
$M \leftarrow \emptyset$
for каждого интервала (i, l_i, r_i) в отсортированном порядке **do**
    Найти минимальное $j \in R$ такое, что $j \geq l_i$
    if j существует и $j \leq r_i$ **then**
        $M \leftarrow M \cup \{(i, j)\}$
        Удалить j из R
    end if
end for
return M

Анализ сложности

- Сортировка левой доли по r_i работает за $O(n \log n)$.
- Для каждой вершины левой доли выполняется запрос в множестве R (например, с помощью сбалансированного двоичного дерева), что занимает $O(\log n)$.
- Таким образом, общий алгоритм выполняется за $O(n \log n)$.

Задача N2

Рассмотрим граф G и игру, в которой из стартовой вершины v_0 игроки по очереди перемещают фишку по ребрам в вершины, ранее не посещавшиеся. Проигрывает тот, кто не может сделать ход.

(а) Доказательство условия выигрышности стартовой вершины

Утверждение. Первый игрок выигрывает, если и только если стартовая вершина v_0 содержится во всех максимальных паросочетаниях графа G.

Доказательство:

(Necessity) Если существует максимальное паросочетание M, в котором v_0 не покрыта, то второй игрок может выбрать ответные ходы, основываясь на структуре M, что приведёт к тому, что первый игрок рано или поздно столкнётся с тупиковой позицией. Таким образом, отсутствие v_0 во *каком-либо* максимальном паросочетании означает, что выигрыш первого игрока нельзя гарантировать.

(Sufficiency) Если v_0 содержится во всех максимальных паросочетаниях, то первый игрок всегда может выбирать такой ход, чтобы в следующей позиции оставался инвариант: независимо от действий противника, существует максимальное паросочетание, покрывающее текущую позицию. Такой выбор обеспечивает, что противник рано или поздно останется без допустимого хода.

Данное доказательство можно формализовать с использованием индуктивного метода по числу оставшихся ходов в игре и анализа альтернативных (чередующихся) путей относительно фиксированного максимального паросочетания.

(б) Алгоритмы определения выигрышности вершин в двудольном графе

Пусть $G = (L \cup R, E)$ — двудольный граф с n вершинами в каждой доле. Требуется для каждой вершины v определить, является ли она выигрышной (то есть, если взять v в качестве стартовой, то первый игрок выигрывает).

Алгоритм со сложностью $O(n^2m)$

1. Вычислите максимальное паросочетание M в графе G.
2. Для каждой вершины $v \in V(G)$ выполните следующие действия:
   (a) Удалите вершину v (то есть, удалите все инцидентные ей ребра) и вычислите максимальное паросочетание M_v в полученном графе $G \setminus \{v\}$.
   (b) Если $|M_v| < |M|$, то v является обязательной (то есть, она входит во все максимальные паросочетания). Следовательно, стартовая позиция в v является выигрышной для первого игрока.

При наивной реализации повторный поиск максимального паросочетания для каждой вершины приводит к общей сложности $O(n^2m)$.

Оптимизация до сложности $O(nm)$

Для ускорения анализа используют структурное разложение, например, разложение Дульмава–Мендельшумана, которое позволяет найти все *обязательные вершины* (критические с точки зрения покрытия максимальными паросочетаниями) за время $O(n+m)$ или $O(nm)$ при обработке всех вершин.

Основная идея:

- Постройте граф альтернативных (чередующихся) путей относительно выбранного максимального паросочетания M.
- Проведите поиск (например, DFS или BFS) в этом графе, чтобы определить, какие вершины можно «заменить» в M. Вершина v является критической (выигрышной стартовой позицией), если она не допускает альтернативной замены, то есть входит во все максимальные паросочетания.

Таким образом, используя данные структуры и объединённый перебор для всех вершин, можно получить маркировку всех вершин за суммарное время $O(nm)$.

Задача N3

Рассмотрим метод **раздвоения вершин** и перевод задачи к поиску **перфектного паросочетания** в двудольном графе.

1. Раздвоение вершин

Для каждой вершины $v \in V$ создадим две вершины:

$$v_{\text{in}}, \quad v_{\text{out}}.$$

Построим двудольный граф $G' = (L, R, E')$, где:

$$L = \{v_{\text{out}} \mid v \in V\}, \quad R = \{u_{\text{in}} \mid u \in V\},$$

и ребро $(v_{\text{out}}, u_{\text{in}}) \in E'$ существует тогда и только тогда, когда в исходном графе G есть дуга $(v \to u) \in E$.

2. Поиск паросочетания

В графе G' ищем **максимальное паросочетание** $M \subseteq E'$, в котором каждая вершина покрыта хотя бы одним ребром.

3. Восстановление циклов

Если найдено паросочетание M, покрывающее все вершины V, то:

- Каждая вершина v участвует ровно в одном исходящем и входящем ребре.
- Это соответствует отображению вершин в циклическое разбиение исходного графа.
- Объединяя такие пары, можно восстановить разбиение на **непересекающиеся простые циклы**, покрывающие все вершины.

Если такое паросочетание отсутствует (т.е. некоторые вершины не покрыты), то искомое покрытие циклами невозможно.

Асимптотика

- Построение G' занимает $\mathcal{O}(n + m)$.
- Поиск максимального паросочетания в двудольном графе — $\mathcal{O}(\sqrt{n} \cdot m)$ (алгоритм Хопкрофта-Карпа).
- Восстановление циклов — $\mathcal{O}(n)$.

Таким образом, общая сложность алгоритма:

$$\mathcal{O}(nm).$$

Задача N4

Обозначим максимальное паросочетание как $M \subseteq E$. Заведём вспомогательный ориентированный граф G_M, называемый **графом возможных чередующихся путей**, построенный на основе текущего паросочетания M.

1. Построение ориентированного графа G_M

- Все рёбра $e = (u, v) \in M$ направим **из правой доли в левую:** $v \to u$.
- Все рёбра $e = (u, v) \notin M$ направим **из левой доли в правую:** $u \to v$.

Таким образом, в G_M можно ходить по "чередующимся путям начиная с не покрытых вершин.

2. Классификация рёбер

Для всех вершин $u \in L, v \in R$:

- Найдём **достижимые вершины** из всех свободных вершин в L по направленным рёбрам G_M — обозначим множество как Reach_L.
- Найдём **обратные достижимые вершины** в R, из которых можно попасть в свободные вершины в R, по рёбрам G_M в обратном направлении — множество Reach_R.

Теперь для ребра $e = (u, v) \in E$ выполняется:

а) Ребро может входить в **какое-либо** максимальное паросочетание, если:

$$u \in \text{Reach}_L \quad \text{и} \quad v \in \text{Reach}_R.$$

б) Ребро входит **во всякое** максимальное паросочетание, если:

$$e \in M \quad \text{и} \quad u \notin \text{Reach}_L \quad \text{и} \quad v \notin \text{Reach}_R.$$

в) Ребро не входит **ни в одно** максимальное паросочетание, если:

$$u \notin \text{Reach}_L \quad \text{или} \quad v \notin \text{Reach}_R.$$

3. Асимптотика

- Построение одного максимального паросочетания — $\mathcal{O}(\sqrt{n} \cdot m)$.
- Обход в ширину по G_M — $\mathcal{O}(n + m)$.
- Классификация рёбер — $\mathcal{O}(m)$.

Итоговая асимптотика — $\mathcal{O}(nm)$, если использовать «наивный» алгоритм поиска паросочетания (поиск в глубину), или $\mathcal{O}(\sqrt{n} \cdot m)$, если использовать Хопкрофта-Карпа.

Задача N5

Решение

1. Представление в виде графа

Построим неориентированный граф $G = (V, E)$, где:

- Вершины V соответствуют свободным клеткам таблицы.
- Рёбра E соединяют соседние по стороне свободные клетки.

2. Сведение к паросочетанию

Рассмотрим следующее:

1. Заметим, что каждый цикл можно представить как последовательность рёбер, образующих чётный цикл (поскольку вершины должны чередоваться).
2. Окрасим клетки в шахматном порядке: пусть левая верхняя клетка будет белой. Таким образом, соседние клетки всегда имеют разный цвет.
3. Получаем двудольный граф, где:
   - Левая доля — белые клетки.
   - Правая доля — чёрные клетки.
4. Каждое ребро — допустимое перемещение между клетками (соседство по стороне).

3. Поиск паросочетания

Попытаемся найти **совершенное паросочетание** (перфектное) между белыми и чёрными вершинами.

- Каждое ребро соответствует соединению двух клеток в пару.
- Если найдётся совершенное паросочетание, то можно объединить пары рёбер в циклы.
- Поскольку цикл должен содержать хотя бы 3 клетки, исключим случаи, где компоненты размера 2 (это будут пары клеток, соединённых между собой и более ни с кем).

4. Доработка и проверка

Алгоритм:

1. Проверим, что количество свободных клеток чётное — иначе невозможно разбить на циклы.
2. Построим двудольный граф по шахматной раскраске.
3. Найдём **максимальное паросочетание**.
4. Проверим:
   - Совершенное ли паросочетание (все вершины покрыты).
   - Ни одна компонента в графе не состоит только из двух взаимно связанных клеток (т.е. образует цикл длины 2).

Асимптотика

- Количество вершин: $\mathcal{O}(nm)$.
- Количество рёбер — максимум 4 на вершину: $\mathcal{O}(nm)$.
- Поиск максимального паросочетания в двудольном графе: $\mathcal{O}(\sqrt{n} \cdot m) = \mathcal{O}(nm)$ с помощью алгоритма Хопкрофта–Карпа.
- Проверка компонент связности и исключения циклов длины 2 — $\mathcal{O}(nm)$.

Итоговая сложность:

$$\boxed{\mathcal{O}(nm)}$$

Задача N6

Идея решения

1. Замечания

- В каждый день выбор — либо играть, либо танцевать.
- Чтобы проверять, что в каждом окне длины k есть хотя бы x игр и y танцев, можно хранить последние k выборов.

2. Динамическое программирование с автоматом состояний

Введём динамику:

$$\mathrm{dp}[i][g][t]$$

— максимальное удовольствие, если мы выбрали первые i дней, среди последних k дней (если $i \geq k$):
- g из них были играми, - t — танцами,
и при этом $g + t \leq k$, а $g + t = \min(i, k)$.
Переход:
Пусть мы на $i + 1$-й день выбираем играть:

$$\mathrm{dp}[i+1][g+1][t'] = \max(\mathrm{dp}[i+1][g+1][t'], \mathrm{dp}[i][g][t] + a_{i+1})$$

А если танцевать:

$$\mathrm{dp}[i+1][g'][t+1] = \max(\mathrm{dp}[i+1][g'][t+1], \mathrm{dp}[i][g][t] + b_{i+1})$$

При этом если $i+1 > k$, то нужно «выкинуть» самую старую активность — перебором всех возможных старых окон (так как $k \leq n$).
Ограничение: при каждом переходе поддерживать инвариант: если $i \geq k$, то в последних k действиях:

$$g \geq x \quad \text{и} \quad t \geq y$$

3. Начальные условия

Для первых $k - 1$ дней никаких ограничений нет. Начинаем с пустого окна:

$$\mathrm{dp}[0][0][0] = 0$$

4. Ответ

Максимальное значение среди всех $\mathrm{dp}[n][g][t]$, удовлетворяющих условиям.

Асимптотика

- Всего состояний: $n \times k \times k = \mathcal{O}(nk^2)$
- На каждое состояние — $\mathcal{O}(1)$ переходов
- Учитывая $k \leq n$, итоговая сложность:

$$\boxed{\mathcal{O}(n^3)}$$